

Deep Reinforcement Learning for Slime Volleyball: A Rainbow + Self-Play Ablation Study

INDENG 242B Final Project

May 9, 2026

Abstract

We study deep RL on Slime Volleyball, a 12-dim, six-action competitive control task. On a Dueling DQN backbone we incrementally add Rainbow components (Double DQN, n -step, PER, C51, NoisyNet) and contrast them with PPO trained via snapshot self-play and a scripted-baseline mixer. Three findings stand out: (i) n -step returns *without* Double DQN are actively harmful (-1.6 reward vs. $+0.6$ when paired with Double); (ii) an entropy-collapsed PPO run is rescued by a four-knob intervention (entropy bonus $10\times$, target-KL early-stop, linear LR anneal, doubled rollout) for a $+3.6$ reward swing; (iii) an obscure environment-baseline bug seeded a co-evolved lineage that beats a correctly-trained agent at $\sim 8:1$ odds despite a strictly degraded training setting.

1 Introduction

Deep RL exposes methodology questions that supervised pipelines do not: non-stationary data, sparse credit assignment, and tight coupling between exploration and the data the model trains on. The same optimizer and loss family produce radically different outcomes once a policy is allowed to choose its own training data.

We study these effects on **Slime Volleyball** (2), a two-player physics game with a 12-dim observation, DISCRETE(6) actions, and episodes of up to 3000 frames. The environment is small enough that a full ablation matrix fits in a weekend’s GPU budget and large enough that subtle algorithmic choices produce statistically meaningful differences. We benchmark every variant via (a) deterministic-greedy reward against the scripted baseline (`eval_R`) and (b) a 1500-match round-robin Elo ladder over 19 trained variants, and we deploy each policy as an ONNX model in a browser playable for end-to-end reproducibility.

Contributions. (1) A $2\times 2\times 2$ ablation over Dueling, Double, and n -step on a shared trunk that exposes a destructive interaction: n -step *without* Double DQN is worse than vanilla DQN. (2) A self-play scaffold combining a FIFO opponent pool (7, 9) with baseline mixing, plus the empirical finding that pure-pool self-play *underperforms* mixing the scripted baseline. (3) A PPO entropy-collapse case study with a four-change rescue ($+3.6$ reward swing). (4) An “ecosystem effect”: a $\times 10$ obs-scale bug in the in-pool baseline produced a lineage of trained agents (Elo ~ 1800) that beat a correctly-trained agent at $\sim 8:1$ in head-to-head play.

2 Problem Setting

MDP. Slime Volleyball is two-player to five lives. We treat it as a POMDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$ from the right side. $\mathcal{S} \subset \mathbb{R}^{12}$ is $[x, y, v_x, v_y]$ for agent, ball, opponent (scaled by $1/10$); $\mathcal{A} = \text{DISCRETE}(6)$ decodes via a fixed table to a (`forward`, `backward`, `jump`) 3-tuple. The two-player API `env.step(a, otherAction=aopp)` draws a^{opp} either from a frozen self-play snapshot or the scripted baseline. Reward $r \in \{-1, 0, +1\}$ is the agent’s life delta per frame, so episode return $\in [-5, +5]$ with ± 5 a sweep and 0 a draw at the 3000-frame cap. $\gamma = 0.99$.

What is “data”? For DQN, a replay buffer of one-step or n -step transitions from 16 parallel `SubprocVecEnv` workers, uniformly sampled (or PER-prioritized in Rainbow variants). For PPO, on-policy rollouts of length $T \times N = 256 \times 16 = 4096$ transitions consumed once per gradient step.

Two RL-specific data issues. Opponent shift: self-play snapshots were trained against still-older snapshots, so bootstrapped Q-targets estimate “the value of s assuming π_{opp} acts like the snapshot.” We mitigate with a slow target ($\tau=0.005$), a FIFO snapshot pool bounding staleness, and a baseline-mixing knob

$p_{\text{base}} \in [0, 1]$ (we vary $\{0, 0.4, 0.7\}$; 0 is catastrophic — see Section 4). **Reward sparsity:** ≤ 10 nonzero events per 3000-frame episode; at $\gamma=0.99$ a reward 100 frames away is attenuated by ~ 0.37 . n -step ($n=3$) pushes gradient onto actual rewards but, as we show, requires Double DQN’s bias correction to be net-positive.

3 Methods

Architectures. All DQN variants share a 12-128-128 ReLU trunk and a Dueling head (10): value $V_\phi : \mathbb{R}^{128} \rightarrow \mathbb{R}^{64} \rightarrow \mathbb{R}$ and advantage $A_\phi : \mathbb{R}^{128} \rightarrow \mathbb{R}^{64} \rightarrow \mathbb{R}^6$, combined as $Q_\theta(s, a) = V_\phi(s) + (A_\phi(s, a) - \frac{1}{|A|} \sum_{a'} A_\phi(s, a'))$ ($\approx 28\text{k}$ params). The plain-DQN cell (**-no-dueling**) collapses both heads into a $128 \rightarrow 64 \rightarrow 6$ MLP. PPO uses a Tanh trunk with orthogonal init, an actor head producing logits $\ell \in \mathbb{R}^6$ (final std 0.01), and a scalar critic head.

Losses. With n -step return $R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k}$, the Double DQN (8) target

$$y_t = R_t^{(n)} + (1 - d_t) \gamma^{n_{\text{actual}}} Q_{\theta-}(s_{t+n}, \arg \max_{a'} Q_\theta(s_{t+n}, a')) \quad (1)$$

is plugged into a Huber loss; **-no-double** reverts to $\max_{a'} Q_{\theta-}$ (vanilla DQN’s positively-biased target). PER (4) adds an importance weight $w_i \propto (1/p_i)^\beta$, $\beta : 0.4 \rightarrow 1.0$. C51 (1) replaces Q with a 51-atom categorical over $[v_{\min}, v_{\max}]$ and recovers expected $Q = \sum_i p_i z_i$ inside **forward** so the ONNX contract stays ($B, 6$). PPO uses the standard clipped surrogate (6) with GAE- λ (5):

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t[\min(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)] + c_v \mathcal{L}_V - c_e \mathcal{H}[\pi_\theta]. \quad (2)$$

Optimization. Adam; DQN uses Polyak target update ($\tau=5 \times 10^{-3}$) and grad-norm clip 10. PPO stacks an entropy bonus c_e , advantage normalization, grad clip 0.5, and a target-KL early-stop ($\delta_{\text{KL}}=0.02$): if $\widehat{\text{KL}}_t = (r_t - 1) - \log r_t$ exceeds the threshold during an epoch, remaining update epochs are skipped. Full per-algorithm hyperparameters in Appendix A.

Beyond-class methodology. **(A) Self-play with FIFO pool + baseline mixing.** Every 100k transitions the online net is snapshotted and pushed into a FIFO pool capped at 30 snapshots. At every episode boundary each parallel env independently draws an opponent: the scripted baseline w.p. p_{base} , otherwise uniformly from the pool — the same scaffolding used at much larger scale by AlphaGo (7) and AlphaStar (9). We find $p_{\text{base}}=0$ catastrophically loses generalization to the baseline even when the pool is seeded with the strongest checkpoint (Section 4); pool quality $>$ baseline quality is a tempting intuition that does not survive the data. **(B) PPO entropy-collapse rescue.** Our first long PPO run terminated at **base_R** = -3.4 despite **pool_R** $\approx +1$; entropy collapsed from $\approx \log 6 \approx 1.79$ to 0.108 by step 1.4M (a $16\times$ drop). The rescue cluster ($c_e : 0.001 \rightarrow 0.01$, rollout $128 \rightarrow 256$, linear LR anneal, target-KL stop) restored $\mathcal{H} = 1.42$ and lifted **base_R** to $+0.2$, a $+3.6$ swing.

4 Results

We evaluate on two axes: **eval_R** (deterministic-greedy reward per episode against the scripted baseline, 5 episodes per checkpoint, $\in [-5, +5]$) and ladder *Elo* from a 1500-match round-robin among 19 trained variants ($\times 2$ clones, $K=32$, init 1500); the scripted RNN baseline is included as a reference player.

4.1 Component ablation: $2 \times 2 \times 2$ over Dueling, Double, n -step

All cells share trunk, target, replay buffer, and 5M-transition budget; only the algorithmic switches differ.

(F1) Dueling alone barely helps (A $\rightarrow$ B: $+0.2$): on a 12-dim observation there are few states where “the action does not matter,” which is exactly the structural signal Dueling exploits. **(F2) Double DQN is the single biggest win** (B $\rightarrow$ C: $+0.6$): with a small/noisy Q-net, $\max_a Q_{\theta-}(s', a)$ is positively biased, and the Double decoupling removes most of that bias. **(F3) n -step without Double is actively harmful** (B $\rightarrow$ D: -1.8): off-policy bootstrap noise compounds across the horizon. Re-introducing Double on top of n -step (D $\rightarrow$ E: $+1.8$) nearly fully restores the deficit — a destructive *interaction*, not a sum of marginal effects, and a fact obscured in the original Rainbow paper (3) which only ablates *removing* components from the full

Table 1: Component ablation. $\text{eval_R} \in [-5, +5]$ is the deterministic-greedy mean over 5 evaluation episodes. “ Δ ” is versus the plain-DQN floor (cell A).

Cell	Dueling	Double	n -step	eval_R	Δ
A	—	—	1	-0.6 ± 0.5	(floor)
B	yes	—	1	-0.4	+0.2
C	yes	yes	1	$+0.2 \pm 0.4$	+0.8
D	yes	—	3	-2.2 ± 1.2	-1.6
E	yes	yes	3	-0.4	+0.2

stack. **(F4)** n -step+Double $\approx n=1$ +Double on this task (C, E within noise); if only one upgrade over plain DQN is affordable, add Double.

4.2 Self-play opponent mixing

Setting $p_{\text{base}}=0$ (**purepool**) collapsed generalization to the baseline: **base_R** dropped to -1.2 ± 0.7 and ladder Elo to 1454 — 225 below $p_{\text{base}}=0.4$ — despite seeding the pool with the strongest checkpoint (Table 2). The scripted baseline plays an RNN-style reflex game the snapshot pool cannot proxy: pool diversity and pool quality are not the same axis.

Table 2: Self-play baseline-mixing ablation, V1 trainer (20M each).

Variant	p_{base}	Pool seed	base_R	Elo
v1_sp_fixed	0.4	clean ckpts	$+0.4 \pm 0.8$	1679
v1_selfplay_seeded (legacy)	0.4	+ buggy ckpts	—	1824
v1_sp_purepool	0.0	+ ladder-king	-1.2 ± 0.7	1454

4.3 PPO entropy collapse and rescue

The failed PPO run collapses from $\mathcal{H} \approx 1.79$ to 0.108 by step 1.4M (a $16\times$ drop), after which the policy is too deterministic to escape its local optimum. The rescue intervention keeps $\mathcal{H} = 1.42$ throughout and lifts **base_R** from -3.4 ± 1.5 to $+0.2 \pm 0.4$ — a +3.6 swing.

4.4 Ladder Elo and eval_R disagreement

Self-play vs. cold (same algorithm) is the largest single gain. **v1_selfplay** (Elo 1795) vs. **v1 cold** (Elo 1282) is a 513-Elo gap, $\sim 94\%$ expected-win. **Full Rainbow with C51+NoisyNet collapsed:** **rainbow_full** (1084) and **rainbow_eps** (1071) lose 95%+ of ladder matches because $[v_{\min}, v_{\max}] = [-5, +5]$ matches episode-return extremes exactly, concentrating mass on the boundary atoms; the simpler “Rainbow-Lite” (Dueling+Double+ n -step+PER) reaches Elo 1690 on the same budget. **eval_R can disagree with Elo:** **plain DQN** (Elo 1520) outranks **dueling** (1453) despite worse **eval_R**, and **ppo_fixed** (failed; Elo 1540) outranks **ppo_rescue** (1497) despite a +3.6 **base_R** advantage — **eval_R** measures performance against one specific opponent, Elo against the entire ecosystem.

Negative results matter. Of five hypotheses we expected to confirm — “Rainbow always helps,” “ n -step is a free win,” “self-play > cold,” “pool quality > baseline quality,” and “**eval_R** predicts Elo” — only *self-play > cold* survived contact with the data.

5 Safety, Robustness, and Ecosystem Effects

Three failure modes from this toy game scale unfavorably to higher-stakes RL deployments.

(a) Adversarial overspecialization. Self-play agents trained against a small pool learn opponent-specific exploits rather than generally robust strategies. Pure-pool agents play strongly against neural-net opponents and weakly against the scripted baseline; baseline-mixed agents are more general-purpose but trade off peak-vs-pool performance.

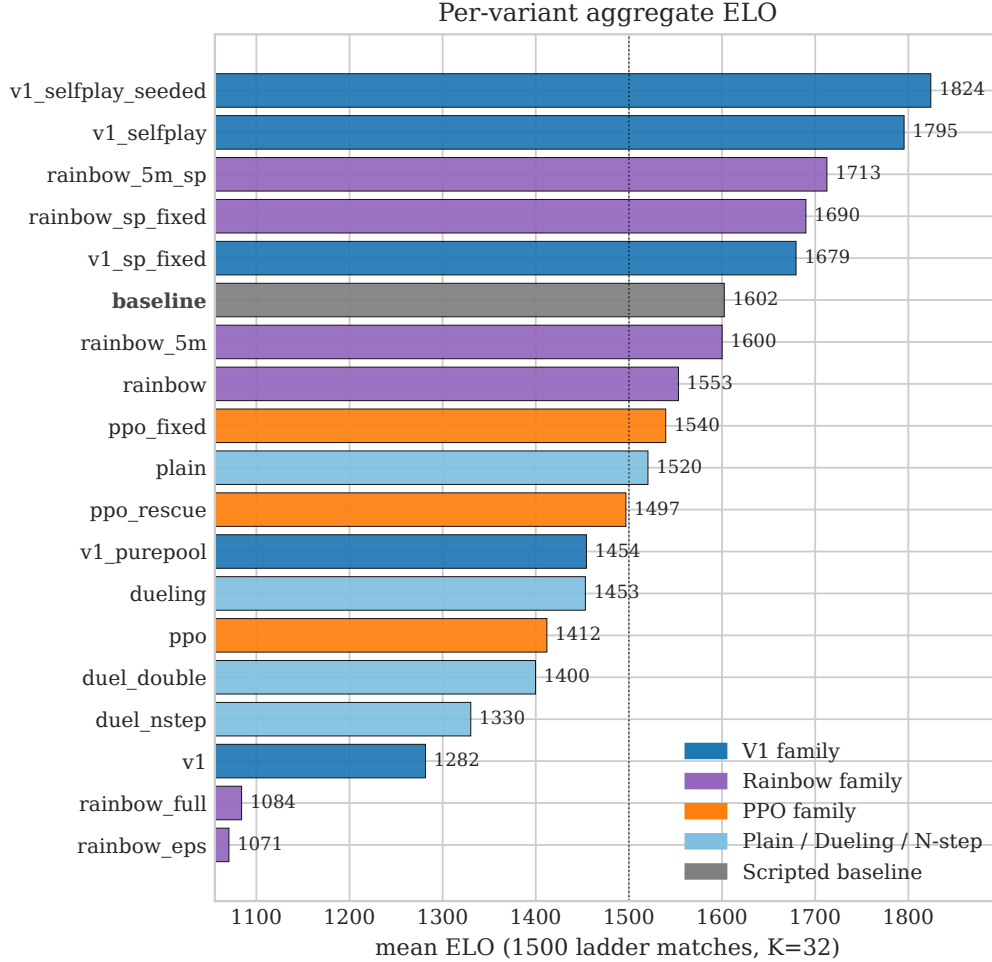


Figure 1: Round-robin Elo over 19 variants (1500 matches). Scripted baseline (highlighted) ranks 6th; Rainbow-with-C51 variants (bottom) collapsed (see text).

(b) Ecosystem effect. The most cautionary finding is the head-to-head between **v1_sp_fixed** (correctly-trained) and **v1_selfplay** (buggy ladder king). Across 200 matches with side-alternation: buggy lineage 71 wins (35.5%), fixed lineage 9 wins (4.5%), 120 time-cap draws (60%) — the buggy lineage wins $\sim 7.9\times$ more often *despite* a strictly degraded training environment (a $\times 10$ obs-scale bug saturated the in-pool baseline’s tanh layer). The buggy lineage co-evolved against itself, so playstyles encode mutual counters; the fixed agent, trained against the *real* baseline, behaves differently and is exploitable. A shared training quirk becomes the local lingua franca, and a corrected replacement model can underperform until the ecosystem re-equilibrates — relevant for autonomous driving fleets, recommendation, and multi-agent market makers.

(c) Reward hacking. PPO reward-hacked a clean, sparse, fully-observed reward via entropy collapse: $\text{pool_R} \approx +1$ while $\text{base_R} = -3.4$. The diagnostic workflow (entropy, KL, clip-fraction monitoring with entropy-bonus and KL early-stop intervention) is the production-realistic mitigation.

Deployment hygiene. Three checks before promoting any self-play-trained policy: (i) evaluate against *multiple* held-out opponent distributions; (ii) rerun head-to-head against the previously deployed version, since a corrected replacement may locally regress; (iii) monitor policy entropy and KL between consecutive snapshots as health metrics, not curiosities.

6 Conclusion

We presented a controlled ablation of deep RL on Slime Volleyball covering Dueling/Double/ n -step DQN, PER, C51, NoisyNet, and PPO with snapshot self-play and baseline mixing. The most informative outcomes were not the “Rainbow works” confirmation we initially aimed for but the negative results: n -step is harmful without Double, pure-pool self-play loses generalization to a stylistically different baseline, full Rainbow-with-C51 collapsed because $[v_{\min}, v_{\max}]$ matched episode-return support, and a fixed-baseline agent loses to a buggy-lineage ecosystem at $\sim 8:1$. The PPO rescue shows that a small, justified config cluster (entropy $\times 10$, target-KL stop, linear LR anneal, doubled rollout) can swing a failed run by $+3.6$ without invoking any new algorithm. The broader message: in deep RL, algorithm choice is intertwined with the training data the algorithm generates for itself, so additive ablations miss destructive interactions, single-opponent evaluations miss ecosystem effects, and “pool diversity” misses opponent *playstyle* diversity.

References

- [1] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 449–458, 2017.
- [2] David Ha. Slime Volleyball Gym environment. <https://github.com/hardmaru/slimevolleygym>, 2020. Apache 2.0; commit accessed 2026-05.
- [3] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 32, 2018.
- [4] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [5] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [7] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [8] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 30, 2016.
- [9] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [10] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1995–2003, 2016.

A Appendix: Hyperparameters

Table 3: Hyperparameters. “DQN-cold” is the 5M ablation budget; “DQN-SP” is the 20M self-play budget; “Rainbow-Lite” is Dueling+Double+ n -step+PER self-play; “PPO-rescue” is the post-fix entropy-stable run.

	DQN-cold	DQN-SP	Rainbow-Lite	PPO-rescue
Total transitions	5M	20M	5M	20M
Parallel envs (N)	16	16	16	16
Learning rate	2.5×10^{-4}	5×10^{-5}	2.5×10^{-4}	3×10^{-4}
LR schedule	const	const	const	linear $\rightarrow 0$
γ	0.99	0.99	0.99	0.99
n -step	3 (1 in cell C)	3	3	—
Batch size	256	256	256	—
Replay capacity	200k	200k	200k	—
Train freq (per worker)	4	4	4	—
Target τ (Polyak)	0.005	0.005	0.005	—
ϵ schedule (start/end/frac)	1.0/0.02/0.30	1.0/0.02/0.30	1.0/0.02/0.30	—
Rollout length T	—	—	—	256
Update epochs	—	—	—	4
Minibatch size	—	—	—	256
Clip ϵ (PPO)	—	—	—	0.2
GAE λ	—	—	—	0.95
Entropy coef c_e	—	—	—	0.01
Value coef c_v	—	—	—	0.5
Target KL	—	—	—	0.02
Max grad norm	10	10	10	0.5
Snapshot every (steps)	—	100k	100k	100k
Pool max size	—	30	30	30
Baseline-opp prob	—	0.4	0.4	0.7
Eval cadence (steps)	25k	25k	25k	25k

B Appendix: Key Code Excerpts

This appendix reproduces the load-bearing code from `train/train_dqn_v1_vec.py` (Dueling forward, Double-DQN target, the `-no-double` ablation switch) and from `train/train_ppo_selfplay_vec.py` (PPO target-KL early-stop). All excerpts are verbatim modulo whitespace; line numbers refer to the source files at the time of submission.

A. Dueling MLP forward (`train_dqn_v1_vec.py`, lines 70–91)

```

1 class DuelingMLPv1(nn.Module):
2     """Dueling MLP: trunk 12->128->128, value head 128->64->1,
3     advantage head 128->64->6, Q = V + (A - mean A)."""
4
5     def __init__(self, obs_dim: int = 12, n_actions: int = 6) -> None:
6         super().__init__()
7         self.trunk = nn.Sequential(
8             nn.Linear(obs_dim, 128), nn.ReLU(),
9             nn.Linear(128, 128), nn.ReLU(),
10        )
11        self.value_head = nn.Sequential(
12            nn.Linear(128, 64), nn.ReLU(), nn.Linear(64, 1),
13        )
14        self.advantage_head = nn.Sequential(
15            nn.Linear(128, 64), nn.ReLU(), nn.Linear(64, n_actions),
16        )
17
18    def forward(self, obs: torch.Tensor) -> torch.Tensor:
19        features = self.trunk(obs)

```

```

20     value = self.value_head(features)
21     advantage = self.advantage_head(features)
22     return value + (advantage - advantage.mean(dim=-1, keepdim=True))

```

B. Double DQN target with -no-double ablation switch (train_dqn_v1_vec.py, lines 508–534)

```

1  with torch.no_grad():
2      if args.no_double:
3          # Vanilla DQN target: max over target net's outputs.
4          next_q = target(batch.next_obs).max(dim=-1).values
5      else:
6          # Double DQN: action SELECTION via online, EVAL via target.
7          next_q_online = online(batch.next_obs)
8          next_actions = next_q_online.argmax(dim=-1, keepdim=True)
9          next_q_target_all = target(batch.next_obs)
10         next_q = next_q_target_all.gather(1, next_actions).squeeze(1)
11
12         # n-step discount: gamma^{n_actual_i} per sample (n_actual=n on full
13         # windows, < n on episode-truncated windows).
14         discount = torch.pow(
15             torch.full_like(batch.rewards, gamma),
16             batch.n_actual.to(batch.rewards.dtype),
17         )
18         td_target = batch.rewards + discount * next_q * (1.0 - batch.dones)
19
20     q_all = online(batch.obs)
21     q_pred = q_all.gather(1, batch.actions.unsqueeze(1)).squeeze(1)
22     loss = F.smooth_l1_loss(q_pred, td_target) # Huber
23
24     optimizer.zero_grad(set_to_none=True)
25     loss.backward()
26     nn.utils.clip_grad_norm_(online.parameters(), max_norm=grad_clip_norm)
27     optimizer.step()
28
29     soft_update(target, online, soft_tau) # tau = 5e-3

```

C. PPO target-KL early stop (train_ppo_selfplay_vec.py, lines 589–675)

```

1  target_kl = args.target_kl if args.target_kl is not None else None
2  early_stopped_epoch = None
3  for _epoch in range(int(args.update_epochs)):
4      np.random.shuffle(indices)
5      # Per-epoch KL accumulator. We decide AT THE END of an epoch
6      # (CleanRL convention) whether to skip the remaining epochs --
7      # checking inside the minibatch loop is too noisy for small mb.
8      epoch_kls = []
9      for start in range(0, total, mb):
10         end = min(start + mb, total)
11         mb_idx = torch.from_numpy(indices[start:end]).to(obs_all.device)
12
13         new_logits, new_values = model.forward_train(obs_all[mb_idx])
14         dist = Categorical(logits=new_logits)
15         new_log_probs = dist.log_prob(actions_all[mb_idx])
16         entropy = dist.entropy().mean()
17
18         log_ratio = new_log_probs - old_log_probs_all[mb_idx]
19         ratio = torch.exp(log_ratio)
20
21         # PPO clipped surrogate
22         unclipped = ratio * mb_advantages
23         clipped = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps) *
24             mb_advantages
25         policy_loss = -torch.min(unclipped, clipped).mean()
26
27         # PP02 clipped value loss + entropy bonus

```

```

27     value_loss = clipped_value_loss(new_values, mb_returns,
28                                     old_values_all[mb_idx], clip_eps)
29     loss = policy_loss + vf_coef * value_loss - ent_coef * entropy
30
31     optimizer.zero_grad(set_to_none=True)
32     loss.backward()
33     nn.utils.clip_grad_norm_(model.parameters(), max_norm=max_grad_norm)
34     optimizer.step()
35
36     with torch.no_grad():
37         approx_kl = ((ratio - 1.0) - log_ratio).mean()
38         epoch_kls.append(float(approx_kl.item()))
39
40     # Early-stop the remaining update epochs if KL overshoots.
41     if target_kl is not None and epoch_kls:
42         mean_kl = float(np.mean(epoch_kls))
43         if mean_kl > target_kl:
44             early_stopped_epoch = _epoch + 1
45             break

```

D. Reproduction commands

The complete experimental matrix can be reproduced with the SLURM scripts in `scripts/`; an A6000 trains a 5M cold cell in ~30 minutes and a 20M self-play cell in ~5 hours.

```

1  # 5M ablation cells (cells A, C, D in Table 2; cells B, E reuse v0/v1 cold)
2  sbatch scripts/sbatch_plain_dqn_5m.slurm
3  sbatch scripts/sbatch_dueling_double_5m.slurm
4  sbatch scripts/sbatch_dueling_nstep_5m.slurm
5
6  # 20M self-play cells
7  sbatch scripts/sbatch_v1_vec_selfplay_fixed.slurm
8  sbatch scripts/sbatch_v1_vec_selfplay_purepool.slurm
9
10 # PPO rescue
11 sbatch scripts/sbatch_ppo_rescue_20m.slurm
12
13 # 1500-match Elo ladder
14 python scripts/ladder_sim.py --n-matches 1500 --output auto
15
16 # Head-to-head (Section 5b)
17 python scripts/head2head.py \
18     --a web/model_v1_selfplay_fixed.onnx \
19     --b web/model_v1_selfplay.onnx --n 200

```